

Лабораторна робота №4. MCP-сервер, мультиагентна конфігурація та доказ користі

25 серпня 2026 р.

Мета

Опублікувати власний MCP-сервер над даними університету, під'єднати його до реального агента, порівняти одиночну й мультиагентну конфігурації на одному наборі задач і показати числами, чи виправдана друга архітектура.

Робота виконується **бригадою з трьох осіб**. Лекції 13, 16, 17.

Усе безкоштовне. MCP SDK відкритий, модель локальна, трасування — OpenTelemetry. Платних сервісів не потрібно.

Склад бригади і розподіл ролей

Кожен учасник відповідає за свою підсистему, має власний результат і захищає його окремо. Спільним є лише інтеграційний крок і підсумковий звіт.

- **Учасник А — MCP-сервер і клієнт.** Пише сервер лише для читання з двома інструментами, тестує обидва рівні помилок, веде аудит-лог і під'єднує сервер до готового агента.
- **Учасник В — конфігурації агентів.** Збирає одиночного агента і supervisor із двома виконавцями на однакових інструментах та однаковій моделі, стежить за ізоляцією контексту й обліком токенів.
- **Учасник С — оцінювання й трасування.** Складає набір задач, рахує pass^k та інтервали, калібрує LLM-суддю і будує траси за конвенціями OpenTelemetry.

Інтеграційний крок виконують разом: сервер учасника А підключається до обох конфігурацій учасника В, після чого набір учасника С проганяється на кожній. Рішення «чи виправдана друга архітектура» ухвалює бригада за числами, а не за враженням.

Теоретичні відомості

Доки кожен застосунок пише власний конектор до кожної системи, кількість інтеграцій росте як добуток $M \cdot N$. Спільний протокол розриває добуток на суму $M + N$. Виграш зявляється не одразу: на малих числах суми й добуток майже збігаються, і накладні витрати протоколу помітніші за економію.

- **Підключення сервера є розширенням межі довіри, а не інтеграцією.** Сервер отримує ваш контекст і повертає текст, який стане для моделі інструкцією. Тому сервер лише для читання пишуть першим, а права додають після того, як запрацював журнал аудиту.
- **Протокол розділяє два рівні помилок.** Невідомий інструмент і аргументи, що не пройшли схему, — протокольні помилки JSON-RPC, корисні застосунку. Семантично хибні дані схему проходять і повертаються як результат із `isError: true`, придатний для самовиправлення моделі. Плутати рівні означає або ховати збій від застосунку, або позбавляти модель зворотного зв'язку.
- **У `stdio`-транспорті `stdout` належить протоколу.** Один `print()` у сервері ламає обмін мовчки — це найчастіша помилка перших годин роботи з MCP. Логи йдуть тільки в `stderr`.
- **Другий агент множить не лише якість.** Разом із ним ростуть канали координації, витрата токенів і кількість режимів відмови. Справжньою причиною виграшу є ізоляція контексту: якщо кожен агент тягне те саме, що бачать інші, витрата росте квадратично, а користі немає.
- **`pass@k` і `passk` рахуються на тій самій множині прогонів і дають протилежні відповіді.** Перша росте з k і описує дослідника, який має право перезапустити; друга спадає і описує продакшн, де спроба одна. Розрив між середньою точністю і надійністю задає *розподіл успіху* за задачами, а не середнє.
- **Інтервал Вілсона описує кожен частку окремо і не є тестом різниці.** Різницю оцінюють парно на тому самому наборі. Три прогони тих самих сорока задач не дають ста двадцяти незалежних спостережень: одиниця ресемплінгу — задача.
- **Суддя на основі моделі має виміряні упередження** — до позиції, до багатослівності й до власного стилю. Звітують не сиру частку збігів із людиною, а капцу: суддя, який завжди каже «А», має високу сиру узгодженість і нульову інформативність.

Корисні посилання для ознайомлення:

- Model Context Protocol: [з чого почати](#) та [Server: Tools](#);

- Довідкові сервери і офіційний реєстр;
- Claude Code: MCP — як підключити сервер до робочого агента;
- Anthropic, [How We Built Our Multi-Agent Research System](#) і Cognition, [Don't Build Multi-Agents](#);
- OpenTelemetry, [GenAI spans](#).

Порядок виконання

Позначка [A], [B], [C] вказує відповідального за крок; [разом] — крок виконує вся бригада.

1. [разом] **Визначити межу сервера:** які дані сервер віддає, які інструменти має і чого не має принципово. Рішення записується у README як контракт довіри.
2. [A] **Написати MCP-сервер** лише для читання, з двома інструментами над корпусом лабораторної 2. Транспорт stdio, схеми аргументів і результату описані явно.
Перевірка: жоден `print()` не йде в `stdout`; логи тільки в `stderr`.
3. [A] **Написати тести схем на двох рівнях:** відсутнє обов'язкове поле і поле неправильного типу дають протокольну помилку JSON-RPC; семантично хибні дані дають результат із `isError: true` та повідомленням, з якого випливає наступна дія; окремий тест — виклик невідомого інструмента.
4. [A] **Підключити сервер до робочого агента:** додати його у власний клієнт і в готовий агент (наприклад, через `.mcp.json` у Claude Code або аналог). Показати аудит-лог: час, інструмент, аргументи, результат, тривалість.
5. [B] **Зібрати дві конфігурації** з однаковими інструментами і однаковою моделлю: одиночний агент і `supervisor` із двома виконавцями (пошук і перевірка цитат).
Тут є спокуса зробити `supervisor` «розумнішим», давши йому кращий промпт. Тоді порівняння втрачає сенс: різнитися має рівно одна річ — топологія.
6. [B] **Виміряти координаційну вартість:** токени на задачу окремо для кожного агента, кількість передавань, час на координацію. Показати, скільки з них припадає на службовий обмін, а не на корисну роботу.
7. [C] **Скласти набір задач:** 40 задач над цими даними з відомими правильними відповідями, серед них не менше десяти, що потребують кількох джерел. Набір фіксується до експериментів.

8. **[C] Прогнати експеримент:** обидві конфігурації по три рази на всіх 40 задачах. Виміряти частку розв'язаних, pass^3 , точність вибору інструментів, токени й затримку, кількість помилок координації.
9. **[C] Порахувати інтервали:** Вілсон описово для кожної конфігурації; висновок — за 95 % інтервалом парної різниці (bootstrap, ресемплінг за задачами). Письмово відповісти, чи накриває інтервал нуль.
10. **[C] Відкалібрувати суддю:** LLM-суддя для оцінки якості відповіді, ручна розмітка 20 випадків, узгодженість у вигляді каппи. Якщо каппа низька — змінити інструкцію судді й повторити.
11. **[разом] Побудувати траси** у форматі JSONL за конвенціями OpenTelemetry: система, операція, модель, вхідні й вихідні токени, ідентифікатор сесії, версія промпта. Показати, як за трасою відновлюється шлях однієї невдалої задачі.
12. **[разом] Ухвалити рішення:** звіт із таблицею порівняння, інтервалом різниці, вартістю координації і явним висновком до 250 слів — чи виправдана мультиагентна конфігурація для вашої задачі.

Відповідь «мультиагентна конфігурація не виправдана» є повноцінним результатом роботи, якщо вона підкріплена інтервалом різниці й таблицею витрат. Саме такий висновок найчастіше й отримують на задачах цього розміру.

Вимоги до задачі та захисту

1. Репозиторій із сервером, клієнтом і обома конфігураціями; **README** з однією командою запуску кожної частини і таблицею «хто що робив».
2. Аудит-лог і приклад траси невдалої задачі з поясненням, на якому кроці все зламалося.
3. На захисті учасник А демонструє сервер наживо і показує обидва рівні помилок; В пояснює, звідки взялася різниця в токенах; С захищає числа й калібрування судді.
4. Бригада разом обґрунтовує рішення про архітектуру.

Контрольні запитання

1. Як протокол перетворює $M \times N$ інтеграцій на $M + N$ і коли це не дає виграшу?
2. Чим ресурс відрізняється від інструмента в термінах ініціативи?
3. Чому один `print()` у сервері ламає `stdio`-транспорт?

4. Який клас помилки має повернути сервер на неіснуючу дату і чому саме такий?
5. Чому підключення MCP-сервера не є перевіркою його безпеки?
6. У чому різниця між $\text{pass}@k$ і pass^k ?
7. Конфігурація А розв'язала 31 задачу з 40, В — 33. Що з цього випливає?
8. Які три зсуви властиві судді на основі моделі й навіщо тут каппа?
9. Які метрики фіксують координаційну вартість мультиагентної системи?
10. Що саме треба записати у трасу, щоб через тиждень відтворити невдалий прогін?

Література

1. Zheng L. та ін. [Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena](#). NeurIPS, 2023.
2. Cemri M. та ін. [Why Do Multi-Agent LLM Systems Fail?](#) arXiv:2503.13657, 2025.
3. Miller E. [Adding Error Bars to Evals](#). arXiv:2411.00640, 2024.
4. Yao S. та ін. [\$\tau\$ -bench: A Benchmark for Tool-Agent-User Interaction](#), 2024.